5-Fold Data Splitting Logic Documentation

Overview

Why Do We Need This?

When building predictive models (like GLMs for insurance pricing), we need to:

1. **Train** the model on one subset of data
2. **Validate** the model on a different subset to tune hyperparameters and prevent overfitting
3. **Evaluate** final model performance on held-out data

The Problem: If we split data randomly at the record level, we risk **data leakage** - where information from the same policy/customer appears in both training and validation sets, artificially inflating model performance.

The Solution: Split data at the **superpolicy level** (a grouping of related records), ensuring all records for a given superpolicy stay together in the same fold.

Why Use an Objective Function?

Not all random splits are equal. Some splits may accidentally put all high-loss policies in one fold and low-loss policies in another, creating **imbalanced folds** that don't represent the overall data distribution.

The objective function ensures balanced folds by:

- Measuring how close each fold's Pure Premium distribution is to the overall average
- Testing hundreds of different random seeds
- Selecting the seed that produces the most balanced split

Why 5-Fold Instead of Train/Valid/Holdout?

Approach	Pros	Cons
Train/Valid/Holdout (60/20/20)	Simple, fast	Only 60% of data used for training; validation results may vary by random chance
5-Fold Cross-Validation	Uses all data for both training and validation; more robust estimates	Takes longer to train (5 models instead of 1)

With 5-fold CV, each record gets to be in the validation set exactly once, and the training set four times - maximizing data usage.

Required Columns

The following columns are needed for the data splitting process:

Identifier Columns

Column	Type	Purpose
`VIN`	string	Vehicle Identification Number (used to create VIN_Date join key)
`Date`	datetime	Policy date (used to create VIN_Date join key)
`superpolicy_id`	integer	**Critical**: Groups records that must stay together in the same fold (prevents data leakage)

Note: VIN + Date are combined to create `VIN_Date = VIN + '_' + DateYMD` (where DateYMD = Date formatted as YYYYMMDD). This serves as the unique record identifier for joining output back to original data.

Earned Exposure Columns (6 coverages)

Column	Type	Purpose
`ee_bi`	float	Earned Exposure - Bodily Injury
`ee_pd`	float	Earned Exposure - Property Damage
`ee_pip`	float	Earned Exposure - Personal Injury Protection
`ee_med`	float	Earned Exposure - Medical
`ee_coll`	float	Earned Exposure - Collision
`ee_comp`	float	Earned Exposure - Comprehensive

Incurred Loss Columns (6 coverages)

Column	Type	Purpose
`incurred_raw_bi`	float	Incurred Loss - Bodily Injury
`incurred_raw_pd`	float	Incurred Loss - Property Damage
`incurred_raw_pip`	float	Incurred Loss - Personal Injury Protection
`incurred_raw_med`	float	Incurred Loss - Medical
`incurred_raw_coll`	float	Incurred Loss - Collision
`incurred_raw_comp`	float	Incurred Loss - Comprehensive

Total: 15 columns needed

Output File

The final output of this process is a file that can be joined back to your original data.

Output Format

Column	Type	Description
superpolicy_id	integer	Superpolicy grouping ID
VIN_Date	string	Unique record identifier (VIN + "_" + YYYYMMDD)
pp_bi	float	Pure Premium - Bodily Injury
pp_pd	float	Pure Premium - Property Damage
pp_pip	float	Pure Premium - Personal Injury Protection
pp_med	float	Pure Premium - Medical
pp_coll	float	Pure Premium - Collision
pp_comp	float	Pure Premium - Comprehensive
fold	integer	Fold assignment (1-5)

Sample Output

```
superpolicy_id,VIN_Date,pp_bi,pp_pd,pp_pip,pp_med,pp_coll,pp_comp,fold
1001,1HGBH41JXMN109186_20230115,125.50,89.30,45.20,12.80,210.45,95.60,1
1001,1HGBH41JXMN109186_20230215,125.50,89.30,45.20,12.80,210.45,95.60,1
1002,2FMDK3KC7BBA12345_20230301,118.75,92.10,48.50,11.20,195.80,88.40,3
```

How to Use

Join this output back to your original data using `VIN_Date` as the key:

```
original_data = pd.merge(original_data, fold_assignments, on='VIN_Date',
how='left')
```

The Pure Premium Vector

What is Pure Premium?

Pure Premium represents the expected loss per unit of exposure:

```
Pure Premium (PP) = Incurred Loss / Earned Exposure
```

6-Dimensional Vector

For each data subset (fold), we calculate Pure Premium for all 6 coverages:

- `pp_bi = incurred_raw_bi / ee_bi`

- `pp_pd = incurred_raw_pd / ee_pd`
- `pp_pip = incurred_raw_pip / ee_pip`
- `pp_med = incurred_raw_med / ee_med`
- `pp_coll = incurred_raw_coll / ee_coll`
- `pp_comp = incurred_raw_comp / ee_comp`

This creates a **6-dimensional vector** that summarizes the loss characteristics of each fold.

Objective Function

Purpose

The objective function measures how well a fold's Pure Premium distribution matches the overall data. **Lower values = better splits**.

Formula

$$\text{Objective} = \Sigma_i \ |PP\_fold_{,i} - PP\_overall_{,i}| \times w_i$$

Where:

- $PP\_fold_{,i}$ = Pure Premium for coverage i in the current fold
- $PP\_overall_{,i}$ = Overall Pure Premium for coverage i (calculated from full dataset)
- w_i = Weight for coverage i = $EE\_i \ / \ \Sigma \ EE$ (normalized earned exposure)

Why Use Weights?

Coverages with higher earned exposure are more important to match correctly. The weighting ensures:

- Large segments (like Collision) have more influence on the objective
- Small segments (like Medical) don't dominate due to noisy Pure Premium estimates

Python Implementation

```
def objectivefunction(fold_pp, overall_stats):
    """
    Calculate objective function value for a data fold.

    Parameters:
    - fold_pp: Series/DataFrame with Pure Premium for each coverage
    - overall_stats: DataFrame with 'mean_pp' and 'norm_ee' (normalized EE
weights)

    Returns:
    - Objective function value (lower is better)
    """
    objective = np.sum(
        np.abs(fold_pp - overall_stats['mean_pp']) *
```

```
    overall_stats['norm_ee']
        )
        return objective
```

Seed and Simulation Loop

Why Run Multiple Simulations?

Different random seeds produce different splits. We want to find the seed that produces the most balanced folds.

Algorithm

```
INPUTS:
- Data with superpolicy_id and loss columns
- N_simulations = number of seeds to try (e.g., 100 or 500)
- N_folds = 5

PROCESS:
FOR SeedNum = 1 to N_simulations:

    1. Set random seed: random.seed(SeedNum)

    2. Get list of unique superpolicy_id values

    3. Shuffle the list using current seed

    4. Divide shuffled list into 5 equal groups (folds)
       - Fold 1: first 20% of superpolicies
       - Fold 2: next 20%
       - ... and so on

    5. For each fold:
       a. Sum earned exposure and incurred loss across all records in that
fold
       b. Calculate Pure Premium for each coverage
       c. Calculate Objective Function (distance from overall mean)

    6. Calculate Average Objective = mean of all 5 fold objectives

    7. Store: {SeedNum, Average_Objective, Time_Elapsed}

END FOR

OUTPUT: Select seed with LOWEST Average Objective
```

Key Implementation Details

1. **Splitting at superpolicy_id level**:

- All records for a superpolicy stay in the same fold
- Prevents data leakage between folds

2. **Random seed reproducibility**:
   - Use `random.seed()` for reproducible shuffles
   - `random.sample(list, len(list))` shuffles without modifying original
3. **Computational optimization**:
   - Calculate full dataset totals once
   - For each fold, only aggregate that fold's records
   - Train set can be computed as: Full - Validation_Fold

5-Fold Cross-Validation Structure

Split Ratios

Fold	Percentage of Data
Fold 1	~20%
Fold 2	~20%
Fold 3	~20%
Fold 4	~20%
Fold 5	~20%

How It Works During Model Training

For each of **5 iterations**:

- **1 fold** = Validation set (~20%)
- **4 folds combined** = Training set (~80%)

Iteration	Validation Fold	Training Folds
1	Fold 1	Folds 2, 3, 4, 5
2	Fold 2	Folds 1, 3, 4, 5
3	Fold 3	Folds 1, 2, 4, 5
4	Fold 4	Folds 1, 2, 3, 5
5	Fold 5	Folds 1, 2, 3, 4

Final model performance = average across all 5 iterations

Implementation Code

Assign Records to Folds

```
import random

def assign_5folds(df, seed, superpolicy_col='superpolicy_id'):
    """
    Assign each record to a fold (1-5) based on superpolicy_id.

    Parameters:
    - df: DataFrame with superpolicy_id column
    - seed: Random seed for reproducibility
    - superpolicy_col: Name of superpolicy ID column

    Returns:
    - DataFrame with new 'fold' column (values 1-5)
    """
    random.seed(seed)

    # Get unique superpolicies and shuffle
    unique_ids = df[superpolicy_col].unique().tolist()
    unique_ids_sorted = sorted(unique_ids)  # Sort first for
reproducibility
    shuffled_ids = random.sample(unique_ids_sorted, len(unique_ids_sorted))

    # Divide into 5 folds
    n_folds = 5
    fold_size = len(shuffled_ids) // n_folds

    id_to_fold = {}
    for i in range(n_folds):
        start = i * fold_size
        end = start + fold_size if i < n_folds - 1 else len(shuffled_ids)
        for sp_id in shuffled_ids[start:end]:
            id_to_fold[sp_id] = i + 1  # Folds numbered 1-5

    df['fold'] = df[superpolicy_col].map(id_to_fold)
    return df
```

Calculate Fold Statistics

```
def calculate_fold_pp(df, fold_num):
    """
    Calculate Pure Premium for a specific fold.
    """
    fold_data = df[df['fold'] == fold_num]

    coverages = ['bi', 'pd', 'pip', 'med', 'coll', 'comp']
    pp_values = {}

    for cov in coverages:
```

```
        ee_col = f'ee_{cov}'
        inc_col = f'incurred_raw_{cov}'
        total_ee = fold_data[ee_col].sum()
        total_inc = fold_data[inc_col].sum()
        pp_values[f'pp_{cov}'] = total_inc / total_ee if total_ee > 0 else
0

    return pp_values
```

Run Simulation to Find Best Seed

```
def find_best_seed(df, n_simulations=100, overall_stats=None):
    """
    Run multiple seeds and find the one with lowest objective.
    """
    results = []

    for seed in range(1, n_simulations + 1):
        df_temp = assign_5folds(df.copy(), seed)

        fold_objectives = []
        for fold_num in range(1, 6):
            fold_pp = calculate_fold_pp(df_temp, fold_num)
            obj = objectivefunction(fold_pp, overall_stats)
            fold_objectives.append(obj)

        avg_obj = sum(fold_objectives) / 5
        results.append({'seed': seed, 'objective': avg_obj})

    # Find best seed
    results_df = pd.DataFrame(results)
    best_seed = results_df.loc[results_df['objective'].idxmin(), 'seed']

    return best_seed, results_df
```

Complete Workflow Summary

1. **Prepare Data**
   - Ensure data has `superpolicy_id` and all 12 EE/incurred columns
2. **Calculate Overall Statistics**
   - Sum EE and incurred for full dataset
   - Calculate overall Pure Premium for each coverage
   - Calculate normalized EE weights
3. **Run Simulation Loop**

- Try 100-500 different random seeds
- For each seed, calculate average objective across 5 folds
- Track results

4. **Select Best Seed**

- Choose seed with lowest average objective

5. **Apply Final Split**

- Use best seed to assign records to folds 1-5
- Save fold assignments

6. **Use in Modeling**

- For each CV iteration, use 1 fold as validation, 4 as training

---

Document created: June 2026